

Claude Code 架构学习

算法小小怪下士

2026 年 4 月 10 日

摘要

本文基于泄露的 Claude Code 源码聚焦于理解 AI 编程 Agent 的四个关键维度：**推理与规划、记忆与上下文、工具使用、感知与安全**，并对技术细节进行大幅精简，旨在快速建立对 Claude Code 系统架构的宏观认知（比如 Less Scaffolding, More Model; Defense in Depth）。

1 从一个宏观框架开始

2023 年之前，AI 对程序员工作方式的影响主要停留在“补全”层面。GitHub Copilot 在光标位置生成代码片段，开发者逐行审阅并决定是否接受。这种交互模式本质上是人类驱动的：开发者负责拆解任务、确定实现路径、编写代码骨架，AI 只在局部细节上提供加速。

2024 年至今，AI 编程工具正在经历从 Copilot 到 Agent 的范式转变。Cursor、Claude Code 等产品不再等待开发者逐行指示，而是接受一个更抽象的目标，然后自主地搜索代码库、阅读文件、编写代码、运行测试、修复错误，直到任务完成或遇到需要人类决策的节点。这个转变的核心在于 Agent 具备了自主的推理-行动循环能力，能够将一个模糊的目标分解为具体的步骤序列，并在执行过程中根据反馈动态调整计划。OpenAI 在一项为期五个月的内部实验中，仅凭 3 名工程师使用 Codex Agent 就生成了约 100 万行代码，工程师的角色从“写代码”转变为“设计环境、描述意图、构建反馈循环”，这个事件提出了 Harness Engineering 的概念。

Harness Engineering 是指为 AI 编程 Agent 构建一套完整的、安全可控的工作环境 with 约束体系，可将其归纳为公式：Harness = Tools + Knowledge + Observation + Action Interfaces + Permission。具体而言，**Tools** 是 Agent 的“手脚”，赋予其文件读写、执行 Shell 命令、访问网络与数据库等直接操作外部世界的能力；**Knowledge** 是 Agent 的“入

职培训手册”，通过产品文档、API 规范与代码风格指南让它理解项目背景与行事准则；**Observation** 是 Agent 的“耳目”，通过 git diff、错误日志或浏览器状态变化让它感知自身动作产生的结果；**Action Interfaces** 则将这些能力封装为标准的 CLI 命令、API 调用或 UI 交互动作，确保指令能够被系统精准执行；最后，**Permission** 是 Agent 头上的“紧箍咒”，借助沙箱隔离、人工审批流程与信任边界划定，在赋予 Agent 强大生产力的同时严防其造成破坏或越权操作。

Claude Code 的整体架构可以自底向上分为六层。简单来说这个架构的工作流程是指，用户发送指令，**Agent 循环**驱动模型思考并调用工具，**上下文管理**负责让模型“记住”关键信息，**权限管线**负责确保操作安全。

用户界面层 (REPL / SDK / 远程)

查询引擎 (会话管理)

Agent 循环

(准备 → 调用模型 → 执行工具 → 重复)

工具系统	权限管线	上下文管理
(原语组合)	(七步判定)	(五层渐进式压缩)

API 通信层 (流式传输、重试)

基础设施层 (状态、配置、MCP)

2 核心维度一：推理与规划——极简的 ReAct 循环

2.1 核心概念：ReAct 范式

Agent 的推理核心遵循 **ReAct (Reasoning + Acting)** 范式，这是一个简单的“思考-行动-观察”循环：思考 (Reasoning)：模型分析当前任务，决定下一步做什么。行动 (Acting)：调用工具（如读文件、运行命令）来执行决定。观察 (Observation)：将工具执行结果反馈给模型，回到步骤 1，直到任务完成。

2.2 Claude Code 的设计选择：Less Scaffolding

与许多复杂的 Agent 框架不同，Claude Code 没有引入复杂的状态机或流程图，而是将全部逻辑浓缩在一个单一的 **ReAct** 循环中。这体现了其核心理念：**信任模型自身的推理能力，而非依赖复杂的代码编排。**

2.3 关键机制

当任务太复杂时，主 Agent 可以派生子 Agent 去独立完成子任务，子 Agent 完成后只返回摘要结果。递归深度限制为 1，防止成本失控。当 API 调用失败时，系统会自动降级到备用模型或切换为非流式模式重试，保证核心流程不中断。

3 核心维度二：记忆与上下文

3.1 核心问题：上下文窗口装不下了怎么办？

Agent 每执行一步，都会产生大量对话历史和工具输出，很快就会塞满模型的上下文窗口。Claude Code 不依赖外部向量数据库，而是在每次循环前执行一套**五层渐进式压缩管线**。

3.2 压缩管线的五个层级（理解其递进关系）

第一层是**工具结果裁剪**：将超长的工具输出（如几千行的日志）保存到磁盘，只留一个文件路径在上下文中。第二层是**历史截断**：直接丢弃很早以前的对话消息。第三层是**微压缩**：精确删除那些不再需要的旧工具结果（如读过的文件内容）。第四层是**上下文折叠**：对一段连续的对话进行摘要，但原始信息仍保留在后台，不破坏主记录。第五层是**自动压缩**。当以上方法都不够时，调用一次模型生成整段对话的摘要，用摘要替换全部历史。

3.3 长期记忆：MEMORY.md 索引机制

除了上下文压缩，Claude Code 还会将关键信息持久化存储为长期记忆。所有记忆均以 Markdown 文件形式保存在本地，并由一个名为 MEMORY.md 的索引文件统一管理。该索引文件本身不包含具体记忆细节，而是作为“目录”通过超链接指向真正的记忆文件，每行格式为 - [标题](文件.md) —— 一句话描述。

长期记忆按用途分为以下几类：用户记忆：构建用户画像，记录其角色、偏好与技术背景，以便在解释概念时采用符合用户习惯的表达方式。反馈记忆：记录用户对工作流的纠正、确认与验证，包括成功与失败的经验、原因及边界条件，确保后续操作保

持风格一致。参考记忆：存储外部资源的查询路径与获取方式（如监控面板地址、项目管理工具入口），仅记忆查找方法而非资源内容本身。

除上述跨会话的长期记忆外，Claude Code 还维护**会话记忆**，专门服务于当前对话的临时上下文，在会话结束后即停止更新。

4 核心维度三：工具使用——少量原语的组合

4.1 核心概念：原语组合优于专用工具

Claude Code 只提供了大约 15 个基础工具（原语），而不是为每个特定任务创建一个专用工具。核心原语包括：Bash（执行命令）、Read（读文件）、Write（写文件）、Edit（编辑文件）、Grep（搜索代码）。这背后的设计哲学是让模型学会像人类程序员一样，组合 Grep + Read + Edit 来完成任何复杂任务。

4.2 关键机制（简略）

当模型一次请求多个工具时，系统会自动判断哪些可以并行执行（如同时读三个文件），哪些必须串行执行（如 Bash 命令通常独占运行）。BASH 工具是最强大也最危险的。Claude Code 使用 tree-sitter 对命令行进行语法树解析，来评估其安全性。

5 核心维度四：感知与安全——纵深防御

5.1 核心概念：权限 pipeline

任何工具在被执行前，都必须通过一个权限检查。这是一个典型的**纵深防御**模型，从全局规则到具体工具，层层设卡。权限检查包括：**全局禁令**：是否在硬性禁止名单里？（如禁止删除 .git 目录）→ 是则直接拒绝。**全局确认**：是否需要强制用户确认？（如修改系统配置）**工具自查**：工具自己判断操作是否危险（如 Bash 命令是否包含 sudo rm -rf）。**交互与护栏**：检查是否需要用户交互，以及是否触及受保护的目录。

Deny 的单调性：Deny 的决定是最终且不可逆的。即使某个插件或钩子认为一个操作“安全”，只要前面任何一层判定为“拒绝”，该操作就无法执行。这保证了安全策略的刚性。

5.2 权限模式

Claude Code 提供了从 plan（只读模式）到 auto（自动批准大部分操作）的多种模式，用户可以根据信任程度自行选择。